First Automation FIN1209: automate one of Lim's rules in R

Write a small R script that reads price files, finds where a rule says to buy or sell, and draws the signals. Then draw a flowchart that explains it.

Student worksheet 100 points | a few hours over a week

Benjamin C. Sotelo | Institute of Accounts, Business and Finance, FEU Manila

Name _____ Section _____ Date _____

What you are making

Read this page first. It is short.

A **trading rule** is a rule that says exactly when to buy or sell and at what price. In Chapter 7 you met several whose prices are exact. In this activity you teach a computer one of them. You will write a small **R script**: you drop price files into a folder, run the script, and it tells you where the rule says to buy or sell, writes the signals into a table, and draws a chart with each signal marked. Add new prices, run it again, and the table and the chart are updated. That is **automation**: the rule is applied the same way every time without you re-reading the chart by eye.

> YOU DO NOT NEED TO KNOW R
>
> Nobody in this class was taught to code, and you are not expected to know how. Part 4 teaches the R you need, one idea at a time, using the very script you will run. Part 5 runs it with a picture of every step. You only start your own script in Part 7, when you have seen all of this work.

What you hand in

FILE	WHAT IT IS
Your R script	One `.R` file that follows the contract in Part 7.
Your flowchart	A picture of how your script works, as a BPMN diagram in a PDF (best), a `.bpmn` file, or a Markdown file with a Mermaid flowchart.

What is in this folder

FILE	WHAT IT IS
First Automation FIN1209.pdf	This handout.
First Automation FIN1209 - Example Script.R	The finished example script. It works.
First Automation FIN1209 - Example Flowchart.pdf / .bpmn	The example's flowchart, as a picture and as a file you can open and edit.
First Automation FIN1209.mp4 and a narration script	A video of Part 5, if the video is in the folder.

WHAT THESE SIGNALS ARE

The output is **the signals of one rule, applied to past prices**. It is not investment advice and it does not say what price will do next. Chapter 1's point still holds: a forecast cannot be checked when it is made, so a signal is only a statement of what the rule says at that moment.

Later. This is the first of two pieces. Later in the semester a bigger version comes, and you may be grouped for it. Everything you build here is the start of that.

1. What to install

Free, and about 30 minutes. If you cannot install, use the browser option.

You need two programs. **R** is the language, and it does the work. **RStudio Desktop** is a friendly window for writing and running R. Install R first, then RStudio.

> NO R PACKAGES
> The example needs **no packages at all**. It uses only what comes with R. You never type `install.packages`. Your own script should not need one either, because the marker's computer will not have it.

W Windows

1. Go to **https://cran.r-project.org/bin/windows/base/** and click the **Download R** link at the top. Open the file you downloaded and click Next through the installer, keeping every default.
2. Go to **https://posit.co/download/rstudio-desktop/**, scroll to the list, and download the **Windows** installer. Open it and click Next through the installer.
3. Open **RStudio** from the Start menu. A window opens with a pane called Console on the left. That is where R talks.

M Mac

1. Go to **https://cran.r-project.org/bin/macosx/**. Click the first `.pkg` under the latest release, the one for your chip (the Apple icon, then About This Mac, tells you whether it is Apple silicon or Intel). Open the file and click Continue through the installer.
2. Go to **https://posit.co/download/rstudio-desktop/**, download the **macOS** installer, open it, and drag RStudio into the Applications folder.
3. Open **RStudio** from Applications. A window opens with a pane called Console on the left.

C If you cannot install anything

Use **Posit Cloud**, which runs RStudio in your browser. Go to **https://posit.cloud**, make a free account, then **New Project**, then **New RStudio Project**. Everything in this handout works the same way. To put the example folder in, use the **Upload** button in the Files pane.

F For the flowchart

Draw it in **bpmn.io** at **https://demo.bpmn.io/**. It runs in the browser, is free, and needs no account. Or install **Camunda Modeler** from **https://camunda.com/download/modeler/**. Both draw BPMN and both save a `.bpmn` file and export a picture. If you would rather use Markdown, **https://mermaid.live** draws a Mermaid flowchart in the browser.

2. The rule the example uses

Larry Williams' Oops, exactly as the Chapter 7 slides give it.

The slide numbers are those of the Chapter 7 student edition deck. Every price the script uses is on one of these slides.

What the slides say

SLIDE	WHAT IT SAYS
194	Williams' Oops starts with a bar that opens beyond the range of the previous bar: an open above the previous high or below the previous low, which the bar may then trade back through. If the new bar opens above the previous bar's high, Lim places a sell-stop entry order at the previous bar's high and a stoploss at the new bar's high. We sell only if price falls back to the previous high.
199	The bullish Oops is the mirror image. If the new bar opens below the previous bar's low, Lim places a buy-stop entry order at the previous bar's low and a stoploss at the new bar's low.
197	If price turned back through the stoploss, the stoploss closes the position, and Lim adds that a stop and reverse order is initiated at the stoploss level, so we would also take the opposite side there.
193	A sell-stop is filled only if price falls to it, a buy-stop only if price rises to it, and a stoploss closes a position that moves against us.
195	Lim says it is best to look for a bullish Oops in an uptrend and a bearish Oops in a downtrend.

The rule as the script reads it

CASE	TEST, BAR I AGAINST BAR I - 1	ENTRY	STOPLOSS	THEN
SELL (bearish Oops)	Open[i] is above High[i-1], and Low[i] is at or below High[i-1]	High[i-1]	High[i]	reverse: BUY at the stoploss
BUY (bullish Oops)	Open[i] is below Low[i-1], and High[i] is at or above Low[i-1]	Low[i-1]	Low[i]	reverse: SELL at the stoploss

The second half of each test is the slides' words made exact: the order is filled only if price comes back to it.

THE SLIDES LEAVE FOUR THINGS OPEN. THESE ARE OUR CHOICES, NOT LIM'S RULE

One bar only. The slides draw the pull-back happening on the same bar as the gap, so the script looks only at that bar. Whether a later bar could fill the order is not stated.

Order inside a bar. A daily bar gives the high and the low but not which came first. The slides draw the high first and then the fall; the script does not need to know.

No trend filter. Slide 195 says to prefer a bullish Oops in an uptrend and a bearish Oops in a downtrend, but the slides do not give a test a computer can run. The script reports every Oops and leaves the trend for you to look at on the chart. Adding a trend test is a decision you would make and explain.

No profit exit. The slides give a stoploss and a stop and reverse, and nothing about taking profit. The script reports the stoploss and the reverse side and nothing else.

3. The contract: what every script must do

Your script is marked by running it on files you have never seen. It can only work if everybody follows this exactly.

Folders and files

WHAT	RULE
Where it runs	The script is run from the folder that holds it. That folder holds a folder called `data`.
Input	Every file in `data` ending `.csv` is one stock. The ticker is the file name without `.csv`, so `KO.csv` is ticker `KO`.
Output folder	`output`. The script makes it if it is not there.
Signals file	`output/signals.csv`, one row per signal, every input file together. If there are no signals it is still written, with only the header row.
Chart files	`output/<ticker>.png`, one per input file, with every signal marked and numbered.
Packages	None. Base R only. No `install.packages`, no `library()` of anything that is not already in R.
Time	Under 60 seconds for ten files of a year each.

The input columns

COLUMN	MEANING
`Date`	Oldest row first. Either `2026-07-02` or `2026-07-02 16:00:00`.
`Open`	The opening price of the bar.
`High`	The highest price of the bar.
`Low`	The lowest price of the bar.
`Close`	The closing price of the bar.
`Volume`	May be there. The example does not use it.

The first row of the file is these names, spelled and capitalised exactly so. One row is one trading day. A file may have other columns; ignore them.

The columns of output/signals.csv, in this order

COLUMN	MEANING	EXAMPLE
`ticker`	The file name without `.csv`	`KO`
`date`	The day the signal happened, as `YYYY-MM-DD`	`2026-07-07`
`side`	`BUY` or `SELL`, capitals	`SELL`
`entry`	The entry price the rule gives	`84.56`
`stoploss`	The stoploss price the rule gives	`85.68`
`reverse`	Optional. The side of the stop and reverse order at the stoploss. Your own extra columns may follow.	`BUY`

The first five columns are required and marked. The rows go ticker by ticker, then date by date. The chart number of a signal is its row number within its ticker.

3b. Where to get price data, and what kind

Checked on the public sites on 4 November 2026. Where a claim needs an account, it says so instead of guessing.

Chapter 1 names the data technical analysis reads: **price, volume and open interest**. For this activity you need one bar per trading day, with six columns.

The data and what each column is for

COLUMN	WHAT IT IS	WHERE THE COURSE'S RULES USE IT
`Date`	The trading day	Puts the bars in order and names each signal.
`Open`	The first price of the day	The Oops tests the open against the previous bar's high or low (slides 194, 199).
`High`	The highest price of the day	The Oops sell-stop entry is the previous high and its stoploss is the new high.
`Low`	The lowest price of the day	The Oops buy-stop entry is the previous low and its stoploss is the new low.
`Close`	The last price of the day	Colours the candle white or black. Other Chapter 7 rules use the close.
`Volume`	Shares traded in the day	The Oops does not use it. Chapter 7 mentions volume in some bar patterns, and Chapter 6 covers it in full. Keep the column if your rule's slides use it.

Open interest is the number of futures contracts still open, so it exists only for futures. A stock has no open interest, and none of the sources below gives one for a share.

The free sources, and what we found

SOURCE	OPEN, HIGH, LOW, CLOSE, VOLUME	ACCOUNT	CSV FILE
Google Sheets, GOOGLEFINANCE	Yes, daily or weekly, all five, with the formula `"all"`. The Help page lists them.	A Google account	Yes: File, Download, CSV. **This handout uses it.** Google Finance has no quote page for Philippine tickers such as BDO, so expect US stocks only.
PSE EDGE (edge.pse.com.ph), the exchange's own site	Today's open, high, low, last price and volume, and the 52-week range, for each Philippine stock. We found no table of past days.	None to read	No download link found. The site says its content is copyrighted.
Yahoo Finance	A history table of date, open, high, low, close, adjusted close and volume for US stocks, readable without signing in. We found no Philippine stocks.	None to read the table	No Download button when signed out. Whether one appears after signing in is not checked.
Investing.com	Not checked: the site refused our automated visit.	Not checked	Not checked
TradingView	Charts Philippine stocks, for example PSE:BDO, with a free public page. Export of the chart data was not checked.	None to view; export needs an account	Not checked, needs an account
Your broker's platform, for example COL Financial	Not checked: it sits behind a login.	Yes, a brokerage account	Not checked

Prices in the **adjusted close** column are corrected for splits and dividends. The example does not use it.

The script works with any source whose CSV has the columns of Part 3. Google Sheets is the one route this handout walks through step by step, because every student already has an account and it exports a CSV in the right layout. If you find Philippine data elsewhere and can save it as a CSV with those column names, it will run.

4a. Get the example prices yourself

The same route you will use for your own task. We give you no price files.

Google Sheets can fetch daily prices with one formula, and it gives exactly the columns the contract needs: Date, Open, High, Low, Close, Volume. You do this once per stock, and you use the same dates as below so your numbers match this handout.

1. Open **https://sheets.new** while signed in to your Google account. A blank spreadsheet opens.
2. Click cell **A1** and type this formula, then press Enter:

CELL A1, FOR KO

```
=GOOGLEFINANCE("NYSE:KO","all",DATE(2026,7,1),DATE(2026,9,30))
```

1. Wait a moment. About 64 rows fill down from A1, with the six column names in row 1. If you see `#N/A`, check the quote marks are straight and the ticker is spelled `NYSE:KO`.
2. Choose **File, Download, Comma-separated values (.csv)**. Your browser saves a file named after the spreadsheet.
3. Rename it **KO.csv** and move it into your `data` folder. The name matters: the script takes the ticker from it.
4. Do the same for the second stock in a new spreadsheet, with `NYSE:F`, and save it as **F.csv** in `data`. Part 5 runs the script on these two files.
5. **Later, for the update in Part 5:** in each spreadsheet change `DATE(2026,9,30)` in the formula to `DATE(2026,10,30)`, download again, and replace the file in `data` (still named KO.csv and F.csv).

CELL A1, FOR F

```
=GOOGLEFINANCE("NYSE:F","all",DATE(2026,7,1),DATE(2026,9,30))
```

> SWAP THE TICKER, NOTHING ELSE
>
> To use another stock, change only the text inside the first quotes of the formula, for example `"NASDAQ:AMD"`. For a Philippine stock, Google Finance has no quote page for them (we checked BDO), so expect `#N/A`: use a US stock, or another source from Part 3b.

A file from the **Download** step may have a time on every date (`2026-07-01 16:00:00`). The contract allows that and the script handles it.

4. R in 14 small steps

One running script that grows. Each step adds exactly one idea. The new lines are highlighted in yellow. Type each step yourself if you can: it is the quickest way to learn.

Before you start, make a folder on your computer called `First Automation FIN1209` and put the folder `data` inside it with the first-run price files KO and F from Part 4a. In RStudio use **File, New File, R Script**, save it in that folder, then choose **Session, Set Working Directory, To Source File Location**. This tells R which folder you are working in, and every `data/KO.csv` below is relative to it. Run a script with the **Source** button at the top right of the script pane, or select lines and press Ctrl+Enter (Cmd+Enter on a Mac).

Each step shows the whole script so far, with this step's new lines highlighted, and then **exactly what R printed** when we ran it. Your output should match.

B1 A named value (a variable)

A variable is a name that holds a value. The arrow `<-` puts the value on the right into the name on the left. Text goes inside quotes. A line that starts with `#` is a comment: a note for people, which R skips.

`print()` shows a value in the Console.

THE SCRIPT SO FAR, STEP B1

```
1 # the file we will read
2 data_file <- "data/KO.csv"
3 print(data_file)
```

R PRINTED

```
[1] "data/KO.csv"
```

B2 Read a CSV file into a table

`read.csv()` opens a CSV file and returns a table, which R calls a data frame. Each row is one trading day (one bar) and each column is one kind of price. `head(bars, 3)` shows only the first three rows.

THE SCRIPT SO FAR, STEP B2

```
1 # the file we will read
2 data_file <- "data/KO.csv"
3 bars <- read.csv(data_file, stringsAsFactors = FALSE)
4 print(head(bars, 3))
```

R PRINTED

```
                 Date  Open  High   Low Close   Volume
1 2026-07-01 16:00:00 81.70 81.98 80.96 81.29 21734800
2 2026-07-02 16:00:00 81.63 84.14 81.40 84.14 18308400
3 2026-07-06 16:00:00 84.10 84.56 82.32 82.96 18992600
```

B3 One column is a list of values (a vector)

`bars$High` is the whole High column. R calls a list of values like this a vector. Square brackets pick a position: `highs[4]` is the fourth value and `highs[1:5]` is the first five.

THE SCRIPT SO FAR, STEP B3

```
1 data_file <- "data/KO.csv"
2 bars <- read.csv(data_file, stringsAsFactors = FALSE)
3 highs <- bars$High
4 print(highs[1:5])
5 print(highs[4])
```

R PRINTED

```
[1] 81.98 84.14 84.56 85.68 84.83
[1] 85.68
```

B4 Cut text down (the date)

The Date column is text and may carry a time, like `2026-07-02 16:00:00`. `substr(text, 1, 10)` keeps only the first ten characters, the day. Assigning to `bars$Date` replaces the column.

THE SCRIPT SO FAR, STEP B4

```
1 data_file <- "data/KO.csv"
2 bars <- read.csv(data_file, stringsAsFactors = FALSE)
3 bars$Date <- substr(bars$Date, 1, 10)
4 print(bars$Date[1:3])
```

R PRINTED

```
[1] "2026-07-01" "2026-07-02" "2026-07-06"
```

B5 Compare two numbers

A comparison such as `>` or `<=` gives `TRUE` or `FALSE`. We use the fourth bar, `i <- 4`. The previous bar is `i - 1`. The first test asks the Oops question: did this bar open above the previous bar's high? The second asks whether price then came back to that high, that is, whether this bar's low is at or below it.

THE SCRIPT SO FAR, STEP B5

```
1 data_file <- "data/KO.csv"
2 bars <- read.csv(data_file, stringsAsFactors = FALSE)
3 bars$Date <- substr(bars$Date, 1, 10)
4 i <- 4
5 print(bars$Open[i] > bars$High[i - 1])
6 print(bars$Low[i] <= bars$High[i - 1])
```

R PRINTED

```
[1] TRUE
[1] TRUE
```

B6 Decide with if

`if (condition) { ... }` runs the lines inside the braces only when the condition is `TRUE`. `&&` means both conditions must be true. `cat()` prints text and values on one line. This is the bearish Oops on slide 194.

THE SCRIPT SO FAR, STEP B6

```
1 data_file <- "data/KO.csv"
2 bars <- read.csv(data_file, stringsAsFactors = FALSE)
3 bars$Date <- substr(bars$Date, 1, 10)
4 i <- 4
5 prev_high <- bars$High[i - 1]
6 if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
7   cat(bars$Date[i], "SELL at", prev_high, "stoploss", bars$High[i], "\n")
8 }
```

R PRINTED

```
2026-07-07 SELL at 84.56 stoploss 85.68
```

B7 Give if a second case (else if)

`else if` is tried only when the first condition is `FALSE`. The bullish Oops on slide 199 is the mirror image: open below the previous low, then price comes back up to it. We change `i` to a bar that opened below the previous low.

THE SCRIPT SO FAR, STEP B7

```
data_file <- "data/KO.csv"
bars <- read.csv(data_file, stringsAsFactors = FALSE)
bars$Date <- substr(bars$Date, 1, 10)
i <- 10
prev_high <- bars$High[i - 1]
prev_low  <- bars$Low[i - 1]
if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
  cat(bars$Date[i], "SELL at", prev_high, "stoploss", bars$High[i], "\n")
} else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
  cat(bars$Date[i], "BUY at", prev_low, "stoploss", bars$Low[i], "\n")
}
```

R PRINTED

```
2026-07-15 BUY at 83.08 stoploss 82.23
```

B8 Repeat for every bar (a loop over rows)

`for (i in 2:nrow(bars))` runs the lines inside the braces once for every bar from the second to the last, with `i` taking each number in turn. We start at 2 because the first bar has no previous bar.

THE SCRIPT SO FAR, STEP B8

```
data_file <- "data/KO.csv"
bars <- read.csv(data_file, stringsAsFactors = FALSE)
bars$Date <- substr(bars$Date, 1, 10)
for (i in 2:nrow(bars)) {
  prev_high <- bars$High[i - 1]
  prev_low  <- bars$Low[i - 1]
  if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
    cat(bars$Date[i], "SELL at", prev_high, "stoploss", bars$High[i], "\n")
  } else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
    cat(bars$Date[i], "BUY at", prev_low, "stoploss", bars$Low[i], "\n")
  }
}
```

R PRINTED

```
2026-07-07 SELL at 84.56 stoploss 85.68
2026-07-13 SELL at 83.74 stoploss 84.68
2026-07-15 BUY at 83.08 stoploss 82.23
2026-07-17 SELL at 84.95 stoploss 85.56
2026-07-22 SELL at 82.4 stoploss 82.92
2026-07-30 BUY at 88.64 stoploss 87.21
2026-08-03 SELL at 88 stoploss 88.48
2026-08-04 BUY at 86.63 stoploss 85.16
2026-08-05 SELL at 86.68 stoploss 87.15
2026-08-06 SELL at 87.15 stoploss 87.97
2026-08-12 BUY at 86.11 stoploss 85.66
2026-08-18 SELL at 88.02 stoploss 88.96
2026-08-26 SELL at 92.08 stoploss 92.21
2026-08-27 BUY at 90 stoploss 88.96
2026-09-01 SELL at 89.53 stoploss 89.78
2026-09-08 BUY at 87.85 stoploss 87.55
2026-09-10 SELL at 88.4 stoploss 88.96
2026-09-15 BUY at 88.87 stoploss 87.8
2026-09-21 SELL at 88.29 stoploss 88.5
2026-09-29 BUY at 86.89 stoploss 86.56
2026-09-30 SELL at 87.28 stoploss 87.52
```

B9 Collect the results in a table

Printing is not enough: we want a table we can save. First make an empty table with named columns. Then each time a signal is found, build a one-row table and add it to the bottom with `rbind()`.

THE SCRIPT SO FAR, STEP B9

```
data_file <- "data/KO.csv"
bars <- read.csv(data_file, stringsAsFactors = FALSE)
bars$Date <- substr(bars$Date, 1, 10)
signals <- data.frame(ticker = character(), date = character(),
                      side = character(), entry = numeric(),
                      stoploss = numeric(), reverse = character(),
                      stringsAsFactors = FALSE)
for (i in 2:nrow(bars)) {
  prev_high <- bars$High[i - 1]
  prev_low  <- bars$Low[i - 1]
  if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "SELL",
                      entry = prev_high, stoploss = bars$High[i],
                      reverse = "BUY", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  } else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "BUY",
                      entry = prev_low, stoploss = bars$Low[i],
                      reverse = "SELL", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  }
}
print(head(signals, 4))
cat(nrow(signals), "signals\n")
```

R PRINTED

```
  ticker       date side entry stoploss reverse
1     KO 2026-07-07 SELL 84.56    85.68     BUY
2     KO 2026-07-13 SELL 83.74    84.68     BUY
3     KO 2026-07-15  BUY 83.08    82.23    SELL
4     KO 2026-07-17 SELL 84.95    85.56     BUY
21 signals
```

B10 Write a CSV file

`write.csv()` saves a table as a CSV file. `dir.create()` makes the output folder first (it does nothing if the folder exists). Open the new file to see the signals.

THE SCRIPT SO FAR, STEP B10

```
data_file <- "data/KO.csv"
bars <- read.csv(data_file, stringsAsFactors = FALSE)
bars$Date <- substr(bars$Date, 1, 10)
signals <- data.frame(ticker = character(), date = character(),
                      side = character(), entry = numeric(),
                      stoploss = numeric(), reverse = character(),
                      stringsAsFactors = FALSE)
for (i in 2:nrow(bars)) {
  prev_high <- bars$High[i - 1]
  prev_low  <- bars$Low[i - 1]
  if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "SELL",
                      entry = prev_high, stoploss = bars$High[i],
                      reverse = "BUY", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  } else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "BUY",
                      entry = prev_low, stoploss = bars$Low[i],
                      reverse = "SELL", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  }
}
dir.create("output", showWarnings = FALSE)
write.csv(signals, "output/signals.csv", row.names = FALSE, quote = FALSE)
cat("Wrote output/signals.csv\n")
```

R PRINTED

```
Wrote output/signals.csv
```

B11 Draw a chart

`png()` opens a picture file, the plot commands draw into it and `dev.off()` closes it. `plot()` makes the empty frame, `segments()` draws each bar's high-to-low line and `rect()` draws its body from the open to the close: white when the bar closed up, black when it closed down. When the picture file is closed R prints `null device 1`. That is normal.

THE SCRIPT SO FAR, STEP B11

```
data_file <- "data/KO.csv"
bars <- read.csv(data_file, stringsAsFactors = FALSE)
bars$Date <- substr(bars$Date, 1, 10)
signals <- data.frame(ticker = character(), date = character(),
                      side = character(), entry = numeric(),
                      stoploss = numeric(), reverse = character(),
                      stringsAsFactors = FALSE)
for (i in 2:nrow(bars)) {
  prev_high <- bars$High[i - 1]
  prev_low  <- bars$Low[i - 1]
  if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "SELL",
                      entry = prev_high, stoploss = bars$High[i],
                      reverse = "BUY", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  } else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "BUY",
                      entry = prev_low, stoploss = bars$Low[i],
                      reverse = "SELL", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  }
}
dir.create("output", showWarnings = FALSE)
write.csv(signals, "output/signals.csv", row.names = FALSE, quote = FALSE)
n <- nrow(bars)
png("output/KO.png", width = 1400, height = 800, res = 120)
plot(NA, xlim = c(0.5, n + 0.5), ylim = range(bars$Low, bars$High),
     xaxt = "n", xlab = "", ylab = "Price", main = "KO")
ticks <- unique(round(seq(1, n, length.out = 8)))
axis(1, at = ticks, labels = bars$Date[ticks], cex.axis = 0.8)
segments(1:n, bars$Low, 1:n, bars$High)
rect(1:n - 0.3, pmin(bars$Open, bars$Close),
     1:n + 0.3, pmax(bars$Open, bars$Close),
     col = ifelse(bars$Close >= bars$Open, "white", "black"))
dev.off()
```

R PRINTED

```
null device
          1
```

B12 Mark the signals on the chart

Chart commands draw on top of what is already there. `points()` puts a triangle at each signal's entry price: red pointing down for SELL, green pointing up for BUY. A short dash marks the stoploss and `text()` writes the signal's number just beyond that dash; it is its row in `signals.csv`.

THE SCRIPT SO FAR, STEP B12

```
data_file <- "data/KO.csv"
bars <- read.csv(data_file, stringsAsFactors = FALSE)
bars$Date <- substr(bars$Date, 1, 10)
signals <- data.frame(ticker = character(), date = character(),
                      side = character(), entry = numeric(),
                      stoploss = numeric(), reverse = character(),
                      stringsAsFactors = FALSE)
for (i in 2:nrow(bars)) {
  prev_high <- bars$High[i - 1]
  prev_low  <- bars$Low[i - 1]
  if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "SELL",
                      entry = prev_high, stoploss = bars$High[i],
                      reverse = "BUY", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  } else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
    row <- data.frame(ticker = "KO", date = bars$Date[i], side = "BUY",
                      entry = prev_low, stoploss = bars$Low[i],
                      reverse = "SELL", stringsAsFactors = FALSE)
    signals <- rbind(signals, row)
  }
}
dir.create("output", showWarnings = FALSE)
write.csv(signals, "output/signals.csv", row.names = FALSE, quote = FALSE)
n <- nrow(bars)
png("output/KO.png", width = 1400, height = 800, res = 120)
plot(NA, xlim = c(0.5, n + 0.5), ylim = range(bars$Low, bars$High),
     xaxt = "n", xlab = "", ylab = "Price", main = "KO")
ticks <- unique(round(seq(1, n, length.out = 8)))
axis(1, at = ticks, labels = bars$Date[ticks], cex.axis = 0.8)
segments(1:n, bars$Low, 1:n, bars$High)
rect(1:n - 0.3, pmin(bars$Open, bars$Close),
     1:n + 0.3, pmax(bars$Open, bars$Close),
     col = ifelse(bars$Close >= bars$Open, "white", "black"))
x <- match(signals$date, bars$Date)
is_sell <- signals$side == "SELL"
mark_col <- ifelse(is_sell, "red", "darkgreen")
points(x, signals$entry, pch = ifelse(is_sell, 25, 24),
       bg = mark_col, col = mark_col, cex = 1.6)
points(x, signals$stoploss, pch = 95, col = mark_col, cex = 2)
text(x, signals$stoploss, labels = seq_len(nrow(signals)),
     pos = ifelse(is_sell, 3, 1), col = mark_col, cex = 0.9, font = 2)
legend("topleft", legend = c("SELL entry", "BUY entry", "stoploss"),
       pch = c(25, 24, 95), col = c("red", "darkgreen", "gray30"),
       pt.bg = c("red", "darkgreen", NA), bg = "white", cex = 0.8)
dev.off()
```

R PRINTED

```
null device
          1
```

B13 Give a block of code a name (a function)

A function packs lines under a name so they can be reused. `name <- function(inputs) { ... }` defines it, and the last value it produces is what it gives back. We make three: `read_bars`, `find_oops` and `draw_chart`. The lines inside are the ones you already wrote; only the packaging is new.

THE SCRIPT SO FAR, STEP B13

```
read_bars <- function(path) {
  bars <- read.csv(path, stringsAsFactors = FALSE)
  bars$Date <- substr(bars$Date, 1, 10)
  bars
}

find_oops <- function(bars, ticker) {
  signals <- data.frame(ticker = character(), date = character(),
                        side = character(), entry = numeric(),
                        stoploss = numeric(), reverse = character(),
                        stringsAsFactors = FALSE)
  if (nrow(bars) < 2) {
    return(signals)
  }
  for (i in 2:nrow(bars)) {
    prev_high <- bars$High[i - 1]
    prev_low  <- bars$Low[i - 1]
    if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
      row <- data.frame(ticker = ticker, date = bars$Date[i], side = "SELL",
                        entry = prev_high, stoploss = bars$High[i],
                        reverse = "BUY", stringsAsFactors = FALSE)
      signals <- rbind(signals, row)
    } else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
      row <- data.frame(ticker = ticker, date = bars$Date[i], side = "BUY",
                        entry = prev_low, stoploss = bars$Low[i],
                        reverse = "SELL", stringsAsFactors = FALSE)
      signals <- rbind(signals, row)
    }
  }
  signals
}

draw_chart <- function(bars, signals, ticker, path) {
  n <- nrow(bars)
  png(path, width = 1400, height = 800, res = 120)
  plot(NA, xlim = c(0.5, n + 0.5), ylim = range(bars$Low, bars$High),
       xaxt = "n", xlab = "", ylab = "Price",
       main = paste(ticker, "- Williams' Oops signals"))
  ticks <- unique(round(seq(1, n, length.out = 8)))
  axis(1, at = ticks, labels = bars$Date[ticks], cex.axis = 0.8)
  segments(1:n, bars$Low, 1:n, bars$High)
  rect(1:n - 0.3, pmin(bars$Open, bars$Close),
       1:n + 0.3, pmax(bars$Open, bars$Close),
       col = ifelse(bars$Close >= bars$Open, "white", "black"))
  if (nrow(signals) > 0) {
    x <- match(signals$date, bars$Date)
    is_sell <- signals$side == "SELL"
    mark_col <- ifelse(is_sell, "red", "darkgreen")
    points(x, signals$entry, pch = ifelse(is_sell, 25, 24),
           bg = mark_col, col = mark_col, cex = 1.6)
    points(x, signals$stoploss, pch = 95, col = mark_col, cex = 2)
    text(x, signals$stoploss, labels = seq_len(nrow(signals)),
         pos = ifelse(is_sell, 3, 1), col = mark_col, cex = 0.9, font = 2)
  }
  legend("topleft", legend = c("SELL entry", "BUY entry", "stoploss"),
         pch = c(25, 24, 95), col = c("red", "darkgreen", "gray30"),
         pt.bg = c("red", "darkgreen", NA), bg = "white", cex = 0.8)
  dev.off()
}

bars    <- read_bars("data/KO.csv")
signals <- find_oops(bars, "KO")
dir.create("output", showWarnings = FALSE)
draw_chart(bars, signals, "KO", "output/KO.png")
write.csv(signals, "output/signals.csv", row.names = FALSE, quote = FALSE)
cat(nrow(signals), "signals\n")
```

R PRINTED

B14 Repeat for every file (a loop over files)

`list.files()` returns the names of the files in a folder. A `for` loop over them runs the three functions once per file, so any CSV you drop into `data` is read. Adding and saving all the signals together is the last piece. This is the finished script, exactly the file you were given.

THE SCRIPT SO FAR, STEP B14

```
# First Automation FIN1209 - Example Script
# Lim's Williams' Oops (FIN1209 Chapter 7, slides 194 to 201, Lim 2016 p. 231)
# as an R script.
# Put CSV files in the data folder, run this script, read output/signals.csv
# and look at the charts in output. Add new rows and run it again.
# These are a rule's signals, not investment advice.

# ---- Settings ------------------------------------------------------------
data_folder   <- "data"
output_folder <- "output"

# ---- Read one CSV file into a table --------------------------------------
read_bars <- function(path) {
  bars <- read.csv(path, stringsAsFactors = FALSE)
  # The Date column may carry a time ("2026-07-02 16:00:00"); keep the day.
  bars$Date <- substr(bars$Date, 1, 10)
  bars
}

# ---- Find the Oops signals in one table ----------------------------------
# Lim p. 231 (slides 194 and 199): a bar that opens above the previous bar's high puts a sell-stop
# at the previous high and a stoploss at the new bar's high (bearish Oops).
# A bar that opens below the previous bar's low puts a buy-stop at the
# previous low and a stoploss at the new bar's low (bullish Oops).
# A stop and reverse order starts at the stoploss level (slide 197), so the
# reverse column names the opposite side, to be taken at the stoploss price.
# OUR OWN CHOICES, not Lim's (the slides leave them open): a signal needs the
# fill on the same bar as the gap; no trend filter; no profit target.
find_oops <- function(bars, ticker) {
  signals <- data.frame(ticker = character(), date = character(),
                        side = character(), entry = numeric(),
                        stoploss = numeric(), reverse = character(),
                        stringsAsFactors = FALSE)
  if (nrow(bars) < 2) {
    return(signals)
  }
  for (i in 2:nrow(bars)) {
    prev_high <- bars$High[i - 1]
    prev_low  <- bars$Low[i - 1]
    if (bars$Open[i] > prev_high && bars$Low[i] <= prev_high) {
      # Gap up, then price falls back to the previous high: the sell-stop fills.
      row <- data.frame(ticker = ticker, date = bars$Date[i], side = "SELL",
                        entry = prev_high, stoploss = bars$High[i],
                        reverse = "BUY", stringsAsFactors = FALSE)
      signals <- rbind(signals, row)
    } else if (bars$Open[i] < prev_low && bars$High[i] >= prev_low) {
      # Gap down, then price rises back to the previous low: the buy-stop fills.
      row <- data.frame(ticker = ticker, date = bars$Date[i], side = "BUY",
                        entry = prev_low, stoploss = bars$Low[i],
                        reverse = "SELL", stringsAsFactors = FALSE)
      signals <- rbind(signals, row)
    }
  }
  signals
}

# ---- Draw the chart with the signals marked ------------------------------
# Black and white candles; a red down triangle is a SELL entry, a green up
# triangle is a BUY entry, the short dash is its stoploss, and the number is
# the signal's row number for this ticker in signals.csv.
draw_chart <- function(bars, signals, ticker, path) {
  n <- nrow(bars)
  png(path, width = 1400, height = 800, res = 120)
  plot(NA, xlim = c(0.5, n + 0.5), ylim = range(bars$Low, bars$High),
       xaxt = "n", xlab = "", ylab = "Price",
       main = paste(ticker, "- Williams' Oops signals"))
  ticks <- unique(round(seq(1, n, length.out = 8)))
  axis(1, at = ticks, labels = bars$Date[ticks], cex.axis = 0.8)
```

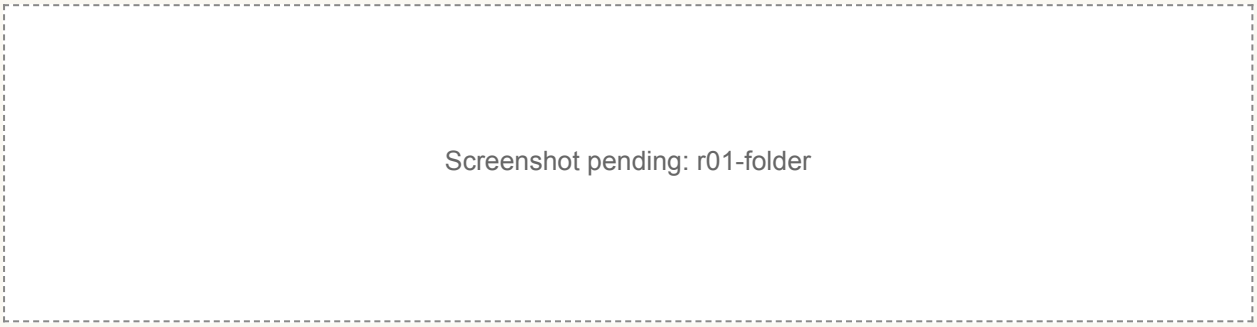
5. Run the finished example

No typing of code. Open it, run it, read the result, then feed it new data and run it again.

1 Set up the folder

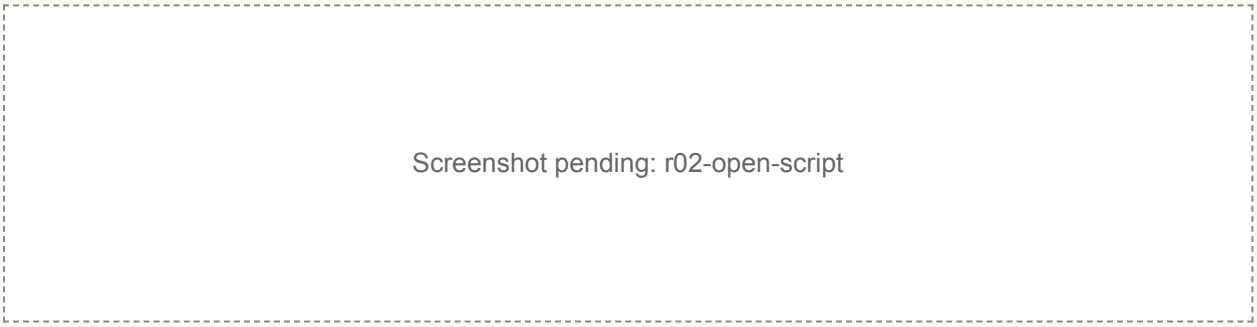
Copy the folder `First Automation FIN1209` from the class Drive to your computer. Inside it are the script, the flowchart files, and two folders of prices. Make a folder called `data` next to the script, and copy the two files `KO.csv` and `F.csv` you downloaded in Part 4a, into it.

The folder now holds the script and a `data` folder with two `.csv` files in it.



2 Open the script in RStudio

In RStudio choose **File, Open File**, and pick `First Automation FIN1209 - Example Script.R`. The script appears in the top-left pane.



3 Point R at the folder

Choose **Session, Set Working Directory, To Source File Location**. The Console prints a `setwd(...)` line. This is the step people forget: without it R cannot find `data`.



4 Run it

Click **Source** at the top right of the script pane. R reads both files, finds the signals, draws two charts and writes the table.

The Console prints one line per file and the name of the file it wrote. On the first-run files that is 21 signals for KO and 12 for F.

Screenshot pending: r04-source

5 Look at what it made

The **Files** pane (bottom right) now has an `output` folder. Open it: `signals.csv`, `KO.png` and `F.png`.

Screenshot pending: r05-output-folder

6 Read the signals

Open `signals.csv` (click it, then View File). Each row is one Oops. `side` says SELL or BUY, `entry` is the price the stop order waits at, `stoploss` is where the trade is closed, and `reverse` is the opposite side Lim starts at the stoploss.

Screenshot pending: r06-signals-csv

7 Read the chart

Open `KO.png`. White candles closed up and black candles closed down. A **red triangle pointing down** is a SELL entry, a **green triangle pointing up** is a BUY entry, the **short dash** beside each is its stoploss, and the **number** is the signal's row for KO in `signals.csv`. Check one: find signal 1 in the table, find the bar with that date, and see that the triangle sits at the entry price.

Screenshot pending: r07-chart

8 Add new data

This is the automation. Close nothing. Copy the two files from the October download (Part 4a, step 6) over the ones in `data`, replacing them. They are the same two stocks with a month of new rows at the bottom.

Screenshot pending: r08-new-data

9 Run it again

Click **Source** again. You change nothing else.

The counts go up: KO from 21 to 28 signals and F from 12 to 18. `signals.csv` and both charts have been rewritten. Open them: the new signals are at the end of the table and the right side of each chart.

Screenshot pending: r09-rerun

10 See the updated chart

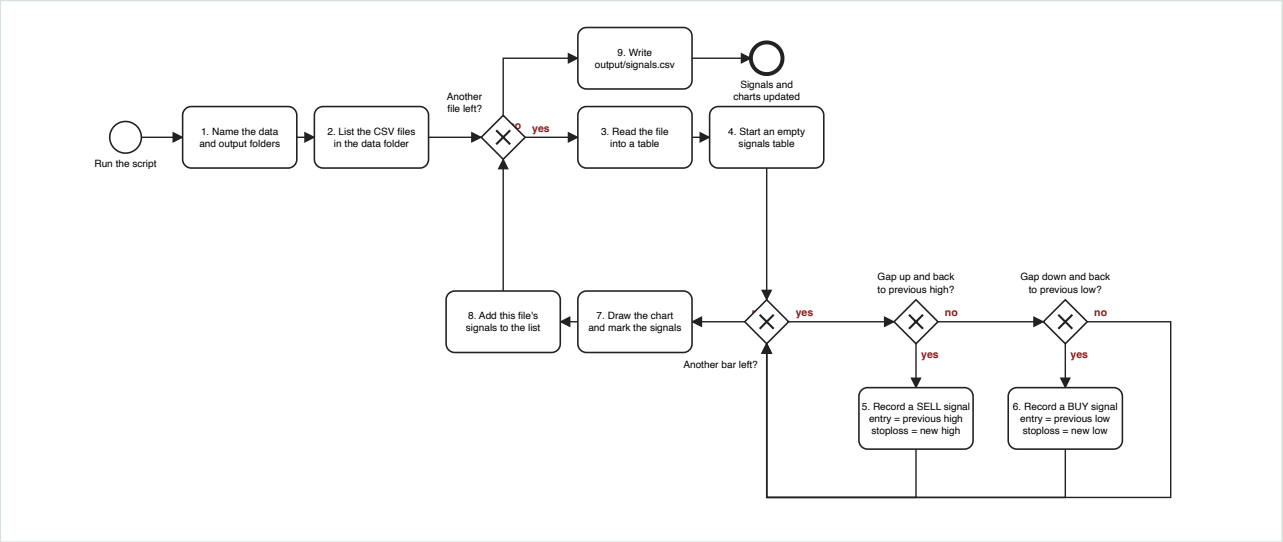
Open `KO.png` again. The chart is longer and the new marks continue the numbering.

Screenshot pending: r10-chart-updated

6. The flowchart of the example

A flowchart says, in shapes, what the script does and in what order. Yours must match your script line for line.

The file `First Automation FIN1209 - Example Flowchart.pdf` is the example's flowchart, drawn in **BPMN**, a standard set of shapes for describing a process. Open `First Automation FIN1209 - Example Flowchart.bpmn` in bpmn.io to see it as a file you can move around. You need only five shapes.



The example script as a BPMN flowchart. The numbers match the table below.

Drawn for this course by build/bpmnkit.py.

The BPMN shapes you need

SHAPE	NAME	USE IT FOR
Thin circle	Start event	Where the script begins. Exactly one.
Thick circle	End event	Where it finishes. Every path must reach one.
Rounded rectangle	Task	One thing the script does, named with a verb: Read the file.
Diamond with X	Exclusive gateway	A yes or no question. Exactly one way out is taken. Label each arrow leaving it, `yes` and `no`.
Arrow	Sequence flow	The order. A loop is an arrow that goes back to a gateway.

Two loops make the whole script. The **outer loop** goes once per file: the first gateway asks whether a file is left, and task 8 sends the flow back to it. The **inner loop** goes once per bar: the second gateway asks whether a bar is left, and every branch sends the flow back to it. Only when no bar is left does the chart get drawn.

Which lines of the example script each shape stands for

SHAPE	SAYS	LINES
Start event	Run the script	the moment you click Source
1	Name the data and output folders	lines 9 to 10
2	List the CSV files in the data folder	line 91
Gateway	Another file left?	line 93
3	Read the file into a table	lines 13 to 18 and line 95
4	Start an empty signals table	lines 30 to 33
Gateway	Another bar left?	line 37
Gateway	Gap up and back to the previous high?	line 40
5	Record a SELL signal	lines 42 to 45
Gateway	Gap down and back to the previous low?	line 46
6	Record a BUY signal	lines 48 to 51
7	Draw the chart and mark the signals	lines 61 to 86 and line 97
8	Add this file's signals to the list	line 99
9	Write output/signals.csv	line 107
End event	Signals and charts updated	line 109

Open the script and read each range beside its shape. Yours should be able to do the same: every shape on your flowchart should point at lines of your script.

Reading it. Start. Name the folders. List the files. Is a file left? If yes, read it, start an empty table, and go to the bars. For each bar, ask whether it gapped up and came back: if yes, record a SELL. If not, ask whether it gapped down and came back: if yes, record a BUY. Either way, go to the next bar. When the bars are done, draw the chart, add the signals to the list, and ask about the next file. When no file is left, write `signals.csv` and end.

7. Your task

Your own script, and a flowchart of it.

Write **your own R script** that applies one rule from Chapters 1 to 7 and meets the contract in Part 3, and draw a **flowchart** of how it works.

Choose your rule

- **Williams' Oops**, as in the example. You may reuse the example's rule, but the script and flowchart must be yours to explain.
- **Hikkake**, which Lim gives with exact prices in Chapter 7 (the end of Part 3 of the slides).
- **Another rule of your choice** from Chapters 1 to 7, as long as the slides or Lim give exact buy and stop prices for it. Write the rule's name and the slide numbers you used in a comment at the top of your script, so the marker can check it against the slides.

> SAY WHAT YOU DECIDED
> Where the slides leave something open, as they did for the example, choose the simplest thing and **say so** in a comment and on your flowchart. Do not bring in an indicator or a definition that is not in the course.

Name your files exactly

FILE	NAME
Your script	`FirstAutomation_Surname_Firstname.R`
Your flowchart, PDF (preferred)	`FirstAutomation_Surname_Firstname_Flowchart.pdf`
Or the .bpmn file	`FirstAutomation_Surname_Firstname_Flowchart.bpmn`
Or a Markdown file	`FirstAutomation_Surname_Firstname_Flowchart.md`

Two files, nothing else. Do not send price files or output: they are rebuilt from your script. If you send a PDF and a .bpmn, send the same flowchart.

Before you submit, check

1. Put your script in a clean folder with a `data` folder of the example files and run it. It must run **with no changes and no installs**.
2. `output/signals.csv` has the five required columns, and one `.png` per input file is in `output`.
3. Delete the `output` folder and run it again. It must come back.
4. Run it with an empty `data` folder. It should not crash, and `signals.csv` should still be written, with only its header.
5. Open your flowchart next to your script. Every shape has lines behind it and every loop and decision in the code is a shape.

> THE MARKER RUNS YOUR SCRIPT ON FILES YOU HAVE NOT SEEN
> Real stocks, a different period, in the layout of Part 3. Some files have `2026-07-02` dates and some have `2026-07-02 16:00:00`. One file may have no signals at all. A script that only works on the example files will not pass.

Getting prices for your own testing. Use Part 4a: the same formula, a different ticker and dates, then File, Download, CSV. Several stocks and several periods make the best test.

8. Using AI

Allowed, with one condition.

You may use AI to help write your script. The condition is that **you can properly visualise and explain it in your flowchart**. The flowchart is where you show that you understand the script, so:

- The flowchart must match **the script you actually submit**, not an earlier version and not what the AI described.
- Each task must say, in your own words, what that part of the code does, and where the rule has a number, which slide it comes from.
- If you cannot explain a part of the script, do not submit it. Ask for a simpler version you can explain, or go back to the example.

A script that runs perfectly with a flowchart that does not match it loses the flowchart marks. A modest script with a flowchart that matches it and explains every step does well.

9. The marks

100 points. 60 are checked against the record by running your script, and 40 are judged from your flowchart.

Checked against the record

WHAT	MARKS	HOW
The script runs	10	On a folder of files you have never seen, with no changes and no installs, in under 60 seconds.
It writes the files the contract names	10	`output/signals.csv` with the required columns (5), and one `output/<ticker>.png` for every input file (5).
It finds the right signals	20	The ticker, date and side of every signal match the record, with none missing and none extra.
It gets the prices right	10	Entry and stoploss match the record, to the cent.
The chart shows them	10	Each chart is a real chart with every signal marked and numbered.
Subtotal	**60**	

Right or wrong. No rubric is needed because the record decides.

Judged, with the course rubric on the next pages

WHAT	MARKS
The flowchart is correct BPMN	10
The flowchart matches your script	15
The flowchart explains each step	15
Subtotal	**40**

NOT MARKED

How your code looks. Neat or messy, short or long, with comments or without: not marked. Only whether it works and whether the flowchart explains it.

YOUR FLOWCHART IS MARKED ON ITS OWN PAGES, AND THE SIGNALS ARE NOT A FORECAST

The script reports what a rule says at a point in the past. Nothing here is marked on whether a signal would have made money.

How a written answer is marked

For the 40 points that a record cannot check. The same rubric marks every written answer in this course.

> READ THIS FIRST
>
> **We are marking whether your answer is grounded and whether it follows, not how long it is.** A long answer that wanders earns less than two precise sentences.

This rubric is for written answers that have no single correct response. Anything that can be checked against the record, a price, a date, a count, is simply right or wrong, and no rubric is needed for it.

What earns the marks

- **Grounded.** Your answer points at something specific and checkable on your own chart: a price, a date, a shape. Not "the market looks bullish", but "the trough on this date, at this price, is higher than the one before it".
- **Connected.** Your conclusion follows from what you pointed at. It is possible to name three real things and then conclude something that has nothing to do with them.

> HONEST: NAMED, NEVER MARKED
>
> A strong answer also names what argues against it. This is what the best answers do, and it is worth learning to do. **No marks are lost for leaving it out.**

The levels

LEVEL	SHARE OF THE MARKS	WHAT IT LOOKS LIKE
Full	100 percent	Grounded and connected.
Partial	75 percent	One of the two is weak: the evidence is vague, or the conclusion does not quite follow from what was named.
Minimal	50 percent	There is a reason, but it is general rather than taken from your own chart.
None	0 percent	A conclusion with no reasoning, or nothing written.

Percentages rather than marks, so the same rubric marks a question worth four points or forty.

The written answers on this sheet, level by level

ANSWER	OUT OF	FULL, 100%	PARTIAL, 75%	MINIMAL, 50%	NONE, 0%
The flowchart is correct BPMN	10	10	7.5	5	0
The flowchart matches your script	15	15	11.25	7.5	0
The flowchart explains each step	15	15	11.25	7.5	0

Every figure in this table is the level's percentage of what the answer is worth, so it cannot disagree with the levels above.

What the rubric looks for in each of them

ANSWER	GROUNDED HERE MEANS	CONNECTED HERE MEANS
The flowchart is correct BPMN	Points at something checkable in your diagram: a start and an end event, a labelled `yes` and `no` leaving every gateway, an arrow back for every loop.	The shapes are used for what they mean: a diamond is a question, a rectangle is an action, and no path stops without an end event.
The flowchart matches your script	Points at the line of your script each shape stands for, and at every loop and decision in the code having a shape.	Following the arrows reproduces what running the script does, in the same order, with no shape for code that is not there.
The flowchart explains each step	Each task says what its lines do, with the actual column, price or file the code uses, and names the slide where the rule has a price.	The explanation tells the reader why that step is there, not only what it is called.

One question, answered at every level

Here is task 5 of the example flowchart explained five ways, one at each level, and two at Partial because there are two ways to be partial. They are written on the example, not on yours.

LEVEL	THE ANSWER	WHY IT SITS THERE
Full, 100%	Task 5, Record a SELL signal: it runs when bar i opened above the previous high and its low came back to that high. It stores entry = High[i-1] and stoploss = High[i], as slide 194 says, and reverse = BUY from slide 197.	Names the exact prices and the slides, and the explanation follows from them.
Partial, 75%	Task 5, Record a SELL signal: it saves the entry price and the stoploss from slide 194 when the gap up fails.	Right idea and the slide, but it never says which price is the entry and which the stoploss, so it cannot be checked.
Partial, 75%	Task 5, Record a SELL signal: it records High[i-1] as the entry and High[i] as the stoploss, because the market is likely to keep falling.	Checkable prices, but the reason given does not follow from them: the code does not predict anything.
Minimal, 50%	Task 5, Record a SELL signal: this is where the program decides to sell, because that is what the rule is for.	A reason, but general: it could describe any task.
None, 0%	Task 5, Record a SELL signal: sell.	A label with no reasoning.

> WHAT THE BEST ANSWERS ADD
>
> A strong flowchart also says where its script is not Lim's rule: "the slides do not say whether a later bar can fill the order, so this script looks at one bar only." It is never marked, and it is the habit that makes a rule you automated one you can trust.